

Data Structure & Algorithm

2021/11/19 ~

박지원

서론

- 'Data Structure And Algorithm 구글 번역'을 읽습니다.
- 그 후에 'Data Structure And Algorithm 원서'의 내용을 다시 읽고 필기합니다.
- 윤성우 자료구조를 참고합니다.
- 자료구조에 대한 깊은 이해를 위해 다시 한 번 공부합니다.
- 구글 번역은 완벽하지 않아 읽기가 쉽지 않고 , 원서의 내용이 윤성우 자료구조와 맥락이 비슷하여 윤성우 자료구조를 기반으로 공부했습니다.

목차

1강_Introduction

2강_Array

3강_Lists , Vector

4강_Stack , Queue , Deque

5강_Map , Set

6강_Tree

<1강_Introduction>

자료구조 data structure이란?

- 자료구조 란 데이터에 편리하게 접근하고 변경하기 위해서 데이터를 저장하거나 조직하는 방법.
- 모든 목적에 맞는 자료구조는 없기 때문에 각 자료구조가 갖는 장점과 한계를 잘 아는 것이 중요하다.

For many problems, the ability to formulate an efficient algorithm depends on being able to organize the data in an appropriate manner. The term *data structure* is used to denote a particular way of organizing data for particular types of operation. These notes will look at numerous data structures ranging from familiar arrays and lists to more complex structures such as trees, heaps and graphs, and we will see how their choice affects the efficiency of the algorithms based upon them.

두 가지뿐이지만, 이 둘을 기준으로 파생되는 이야기가 적지 않으니 약간의 각오가 필요하다. 그리고 자료구조를 공부한다고 하면, 보통 선형 자료구조인 리스트와 스택, 큐 정도까지는 잘 공부한다. 하지만 그 이후에 등장하는 비선형 자료구조에서 무너지는 경우가 허다하다. 여러분은 비선형 자료구조까지 공부할 수 있기를 바란다. 필자가 이 책에서 추구하는 바가 바로 그것이니 말이다.

실무에서는 본서에서 설명하는 자료구조를 직접 구현하지 않고 검증된 라이브러리를 가져다 쓴다. 그리고 그것은 합리적인 선택이다. 검증된 라이브러리를 활용하는 것이 안전성에서나 성능 면에서나 우월하기 때문이다.

하지만 라이브러리를 잘 가져다 쓰려면 리스트가 무엇이고 트리가 무엇인지 알아야 한다. 그냥 아는 것이 아니라 각각의 특성을 정확히 이해해야 한다. 친구들의 주소와 전화번호 정보를 저장하기 위한 자료구조를 선택한다고 가정해보자. 이는 리스트로도 가능하고 트리로도 가능하다. 그렇다면 여러분은 무엇을 선택하겠는가?

선택을 하려면 각각의 자료구조를 잘 이해하고 있어야 한다. 선택을 하는데 있어서 자료구조의 구현 능력은 그리 중요하지 않을 수 있다.

“그럼 각종 자료구조를 가져다 쓸 수 있을 정도로만 이해하면 되는 거 아니에요?”

전혀 틀린 말은 아니다. 코드 레벨에서 이해하지 않아도, 그림의 형태로 자료구조를 이해하고 설명할 수 있을 정도만 되어도 큰 의미가 있다. 하지만 코드 레벨에서 자료구조를 구현한 경험이 있다면 자료구조를 더 잘 알게 된다. 자료구조를 보는 깊이가 달라진다고 할 수 있다.

뿐만 아니라, 자료구조의 구현 경험은 여러분의 프로그래밍 능력을 향상시켜 줄 것이다. 혹시 아직 C언어에 익숙지 않은가? 이 책을 끝까지 공부하고 나면 C언어에 능통하게 될 것이다. 사실 이 책을 공부하는 동안 프로그래밍 실력이 향상되지 않으면 이 책을 끝까지 공부하는데 무리가 따를 것이다.

- 말은 안 했지만 사실은 자료구조를 공부한 경험이 있다.
- 자료구조의 학습경험은 없지만 프로그래밍 경력이 어느 정도 된다.
- 천재적인 두뇌의 도움으로 컴퓨터에 관한 학습속도가 원래 빠르다.

따라서 여러분이 위의 세가지중 한가지에 해당하지 않는다면, 필자가 자료구조를 처음 공부할 때 들었던 시간과 노력의 분량만큼을 여러분도 쏟아 부어야 합니다. 하지만 자료구조를 공부하는 것에는 그만한 가치가 있습니다. 자료구조를 공부하게 되면 프로그래머로서 운영체제를 깊이 있게 이해할 수 있는 기반이 갖춰집니다. 만약에 여러분이 운영체제의 커널이나 오픈 소스에 관심이 있다면 자료구조를 피해야 안 됩니다. C++을 좋아하고 STL과 같은 라이브러리를 효율적으로 활용하고자 한다면, JAVA의 Object 클래스의 멤버인 hashCode 메소드를 제대로 이해하고 활용하고 싶다면 자료구조에 대한 이해가 필수입니다.

조바심을 가질 필요가 없습니다.

저 역시 자료구조를 공부하는데 오랜 시간이 걸렸습니다. 그리고도 수차례 반복해서 공부했던 대표적인 과목이 바로 자료구조입니다. 그러니 한번 시작하면 끝장을 봐야 한다는 각오도 조금은 내려놓으시기 바랍니다. 완벽히 이해하는데 1년이 걸려도 좋고 2년이 걸려도 좋습니다. 꾸준히 공부할 자신만 있다면, 기분 좋은 날만 골라서 공부해도 좋습니다. 하지만 본서를 택하였으니 제가 여러분에게 전하고자 하는 내용을 모두 이해했으면 좋겠습니다. 되돌아 보면 저 역시 자료구조를 강의하는 수준에 이르기까지 대학을 졸업한 후로도 몇 년이 더 걸린 것 같습니다. 늦은 걸음에 조바심을 낼 필요가 없습니다. 제가 증인입니다.

끝으로 아빠가 쓴 책의 내용은 모르지만 표지만 봐도 반가워하는 딸과 아들, 나의 부족한 부분을 채워주고 인내해주는 아내에게 고맙다는 말을 전하고 싶습니다. 그리고 머리말의 끝에 꼭 넣는 '하나님 감사합니다'에 대한 이런저런 얘기를 들긴 하지만 그래도 집필과정을 되돌아 보면 어쩔 수 없습니다. 여전히 제게는 하나님이 유일한 '스페셜 땡스 투'입니다.

<STL 자료구조 분류>

Sequence containers

Sequence containers implement data structures which can be accessed sequentially.

<code>array</code> (C++11)	static contiguous array (class template)
<code>vector</code>	dynamic contiguous array (class template)
<code>deque</code>	double-ended queue (class template)
<code>forward_list</code> (C++11)	singly-linked list (class template)
<code>list</code>	doubly-linked list (class template)

Associative containers

Associative containers implement sorted data structures that can be quickly searched ($O(\log n)$ complexity).

<code>set</code>	collection of unique keys, sorted by keys (class template)
<code>map</code>	collection of key-value pairs, sorted by keys, keys are unique (class template)
<code>multiset</code>	collection of keys, sorted by keys (class template)
<code>multimap</code>	collection of key-value pairs, sorted by keys (class template)

Unordered associative containers

Unordered associative containers implement unsorted (hashed) data structures that can be quickly searched ($O(1)$ amortized, $O(n)$ worst-case complexity).

<code>unordered_set</code> (C++11)	collection of unique keys, hashed by keys (class template)
<code>unordered_map</code> (C++11)	collection of key-value pairs, hashed by keys, keys are unique (class template)
<code>unordered_multiset</code> (C++11)	collection of keys, hashed by keys (class template)
<code>unordered_multimap</code> (C++11)	collection of key-value pairs, hashed by keys (class template)

Container adaptors

Container adaptors provide a different interface for sequential containers.

<code>stack</code>	adapts a container to provide stack (LIFO data structure) (class template)
<code>queue</code>	adapts a container to provide queue (FIFO data structure) (class template)
<code>priority_queue</code>	adapts a container to provide priority queue (class template)

- multiset은 `#include <set>`을 선언하면 사용할 수 있습니다.
- multimap은 `#include <map>`을 선언하면 사용할 수 있습니다.
- unordered_multiset은 `#include <unordered_set>`을 선언하면 사용할 수 있습니다.
- unordered_multimap은 `#include <unordered_smap>`을 선언하면 사용할 수 있습니다.

- Reference : <https://www.cplusplus.com/reference/stl/>

<2강_Array>

1. C#에서 배열은 왜 Heap에 저장될까?

The array is allocated on the heap. But really, you can't think of the .NET heap the same way as in C or C++. Allocating a new object on the heap CAN be, but not always, faster than allocating memory on the stack in .NET.

When a .NET application starts, a chunk of memory is allocated as the heap. So the memory is already allocated, when an object is created with the 'new' keyword, that object is given preallocated memory on the .NET heap.

2. 배열의 선언 방식에 따른 메모리 위치

- A **local array** will be (usually) created on **stack** while
- A **global or static array** will be (usually) created on **bss/data segments** and
- A **dynamically created array** will be created on **heap**.

3. 배열은 메모리에 연속적으로 데이터가 저장됩니다. (stack , data , heap 영역에서 모두 마찬가지)

4. index의 기준이 1인 문제

- index를 1부터 주어주는 문제가 종종 존재합니다. 그 때 vector를 사용할 경우 v[0]에 쓰레기 값을 넣어주고 복잡해지기 때문에 arr를 이용하고 arr[1]부터 문제 전체를 생각합니다.
- 구현은 결국 흐름이기 때문에 문제에서 주어진 대로 구현하면 됩니다.

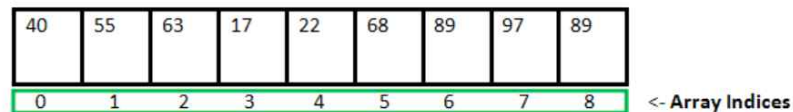
<3강_Lists , Vector>

1. Array vs Linked-List

*** Array와 Vector는 완벽히 동일하지는 않지만 동일하게 생각해도 무방합니다. ***

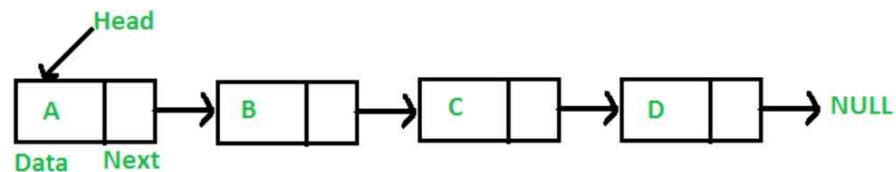
1) 정의

Arrays store elements in contiguous memory locations, resulting in easily calculable addresses for the elements stored and this allows faster access to an element at a specific index. Linked lists are less rigid in their storage structure and elements are usually not stored in contiguous locations, hence they need to be stored with additional tags giving a reference to the next element. This difference in the data storage scheme decides which data structure would be more suitable for a given situation.



Array Length = 9
First Index = 0
Last Index = 8

Data storage scheme of an array



2) 차이점

Major differences are listed below:

- **Size:** Since data can only be stored in contiguous blocks of memory in an array, its size cannot be altered at runtime due to the risk of overwriting other data. However, in a linked list, each node points to the next one such that data can exist at scattered (non-contiguous) addresses; this allows for a dynamic size that can change at runtime.
- **Memory allocation:** For arrays at compile time and at runtime for linked lists. but, a dynamically allocated array also allocates memory at runtime.
- **Memory efficiency:** For the same number of elements, linked lists use more memory as a reference to the next node is also stored along with the data. However, size flexibility in linked lists may make them use less memory overall; this is useful when there is uncertainty about size or there are large variations in the size of data elements; memory equivalent to the upper limit on the size has to be allocated (even if not all of it is being used) while using arrays, whereas linked lists can increase their sizes step-by-step proportionately to the amount of data.
- **Execution time:** Any element in an array can be directly accessed with its index; however in the case of a linked list, all the previous elements must be traversed to reach any element. Also, better cache locality in arrays (due to contiguous memory allocation) can significantly improve performance. As a result, some operations (such as modifying a certain element) are faster in arrays, while some others (such as inserting/deleting an element in the data) are faster in linked lists.

- Array는 연속적으로 데이터를 저장하지만 , Linked-List는 흩어져서 데이터를 저장하고 포인터로 연결합니다.

- 포인터를 저장하기 때문에 단순히 생각하면 Linked-List가 메모리를 더 많이 사용할 것 같지만 , 개수가 자주 변하는 데이터의 경우 Array는 과하게 많은 메모리를 할당하지만 Linked-List는 1대1로 메모리를 할당하기 때문에 더 유연합니다.

- Array는 index를 통해 원소를 바로 찾을 수 있으나 , Linked-List는 traverse(횡단)해야 합니다.

- Array의 연속적인 성질 때문에 캐시에서 유리합니다.

3) Array의 메모리 효율성이 떨어질 수 있는 이유

- **v.capacity();**

- 할당된 공간의 크기를 리턴한다.
- 공간 할당의 기준은 점점 커지면서 capacity를 할당하게 됩니다.
- dev c++ 기준으로 string 클래스로 vector 를 만들었을때.

원소 갯수 1 => capacity 1
원소 갯수 2 => capacity 2
원소 갯수 3 => capacity 4
원소 갯수 4 => capacity 4
원소 갯수 5 => capacity 8
원소 갯수 6 => capacity 8
원소 갯수 7 => capacity 8
원소 갯수 8 => capacity 8
원소 갯수 9 => capacity 16

을 보서는 ~~2^n으로 capacity의 메모리 할당이 되지 않나~~ 싶습니다.(아래에서 테스트 해보겠습니다)

- **기존 메모리의 * 2 로 증가하게 됩니다.**
- 이런식으로 **메모리 할당을 하는 이유**는 push_back이 일어날때 마다 동적할당을 하면, 비효율적이므로 미리 정해진 만큼 동적할당을 한번에 하는 것 입니다.
- 원소 개수가 증가하면서, 메모리를 증가해서 따로 할당하게 되면 기존위치에 메모리를 이어서 할당하는게 아니라 새롭게 메모리를 할당해서 원소들을 전부 복사하는 형태인데요.
기존에는 복사에 대한 비용이 많이 들었습니다. 하지만, std::move 라는 것이 도입되면서 복사하지 않고 이동하게 되어 메모리 증가에 따른 비용이 많이 들지 않게 되었습니다.

- 기존의 Array 나 Vector에 할당된 메모리 보다 많은 데이터가 저장되어야 한다면 미리 동적 할당을 하기 때문에 비효율성이 발생합니다.

2. Vector를 미리 할당 하는 3가지 기술

1) 초기화

```
//1-1. initialize default type
vector<int> v;
for (int i = 0; i < 10000; i++)
{
    v.push_back(1);
}
cout << "size & capacity : " << v.size() << " " << v.capacity() << "\n"; // 10,000 / 12,128

//1-2. initialize parameter type
vector<int> vector_initialize(10000, 1);
cout << "size & capacity : " << vector_initialize.size() << " " << vector_initialize.capacity() << "\n"; // 10,000 / 10,000
```

- 첫 번째 방식은 capacity 자체를 추가로 더 할당해줍니다.
- 두 번째 방식은 size == capacity로 합니다. 만약 10001번째 원소를 추가하려면 capacity를 늘리는 오버헤드가 발생합니다.

2) reserve

```
//2. reserve
vector<int> vector_reserve;
vector_reserve.reserve(10000);
cout << "size & capacity : " << vector_reserve.size() << " " << vector_reserve.capacity() << "\n"; // 0 / 10,000
/*vector_reserve[100] = 123;
cout << vector_reserve[100];*/ //ERROR CODE
```

- capacity가 증가되는 것을 보면 메모리는 할당하지만 초기화되지 않은 상태로 둡니다.
- size 자체는 0이므로 값 할당은 불가능합니다.

3) resize

```
//3. resize
vector<int> vector_resize;
vector_resize.resize(10000);
cout << "size & capacity : " << vector_resize.size() << " " << vector_resize.capacity() << "\n"; // 10,000 / 10,000
vector_resize[100] = 123;
cout << vector_resize[100]; // 123
```

- 메모리도 할당하고 , size도 할당되므로 값을 넣을 수 있습니다.

<4강_Stack , Queue , Deque>

1. Stack & Queue

- 자료구조의 모든 원소를 나열하려면 pop()을 통해 빼고 top() OR front() OR back()으로 확인하는 방법 밖에 없습니다.
- 다시 말해 , range based for문으로 원소를 나열 할 수 없습니다.

```
cout << "Show all queue elements : ";  
showAll(q);
```

```
void showAll(queue <int> q)  
{  
    while (!q.empty())  
    {  
        cout << q.front() << ' ';  
        q.pop();  
    }  
}
```

<Stack과 Queue에서 실수하기 좋은 자료구조 전체 원소 탐색>

```
while (q3.empty() == false)  
{  
    cout << q3.front() << ' ';  
    q3.pop();  
}  
  
for (int i = 0; i < q3.size(); i++)  
{  
    cout << q3.front();  
    q3.pop();  
}
```

- 위의 반복문을 사용 하면 올바르게 Queue의 모든 원소를 탐색합니다.

- 아래의 반복문을 사용 하면 q3.size()가 계속 감소하기 때문에 반복문이 의도한 대로 동작하지 않아 모든 원소를 탐색하지 못합니다.

2. Deque (=Double Enabled Queue)

= Queue와 Vector의 단점을 보완해서 만든 Sequence 자료구조

- 간단한 구조가 요구되는 문제에서는 Vector OR Queue를 사용하지만 , 조금 더 복잡한 경우 완벽한 상위호환인 Deque을 사용합시다.

ex) {1,2,3,4,5}를 Queue로 만들면 push와 pop을 통해 오른쪽으로 밀 수 있지만(ex : 5,1,2,3,4) 왼쪽으로는 밀 수 없습니다. 그러나 Deque은 양방향 모두 가능합니다.

- Deque의 가장 큰 장점은 Vector와 90% 이상 method가 같으면서 Queue의 기능이 추가된 것 입니다. (Vector 앞부분에 push , pop 가능)

```
deque<int> d;
for (int i = 0; i < 5; i++)
{
    d.push_back(i);
}

d.pop_front(); //앞에 숫자(0)를 제거
d.pop_back(); //뒤에 숫자(4)를 제거
d.push_front(7); //앞에 7을 넣기
d.push_back(7); //뒤에 7을 넣기

for (auto i : d)
{
    cout << i << " ";
}
```

7 1 2 3 7

3. priority_queue

1) STL

- priority_queue는 #include <queue> 헤더에 존재합니다.
- queue와 사용방식은 비슷하나 queue에서 pop 되는 순서를 개발자가 직접 지정할 수 있습니다.

2) 기본

```
priority_queue<int> pq;  
priority_queue<int, vector<int>, greater<int>> pq2;  
priority_queue<int, vector<int>, less<int>> pq3;
```

- 간단한 선언을 통해 작은 순서, 큰 순서는 해결할 수 있습니다.

3) 사용자 지정 순서는 Deque으로 해결합니다.

```
priority_queue<int, vector<int>, Cmp> pq4;
```

```
sort(d.begin(), d.end(), Cmp);
```

- 만약 사용자가 지정한 순서대로 담는 자료가 나온다면 Cmp 같은 함수를 만들어서 해결 할 수 있을 것 같지만 priority_queue에서는 그 방식이 복잡하고 실전에서 사용하지 못할 것 같습니다.
- 그러므로 deque을 이용합시다. deque은 기존처럼 사용자 지정 함수에 대한 정렬이 자유롭습니다.
- 만약 priority_queue 대신 deque을 사용해서 풀지 못하는 문제를 발견하면 현재 문서를 개선합니다.

<5강_Map , Set>

1. Map

*** Set은 Map과 동일하나 value가 없는 차이만 존재하기 때문에 필기 하지 않았습니다. ***

1) Map 선언

```
//0. map 선언 (template에 pair를 쓰지 않습니다.)  
map<string, int> m;
```

2) Map 데이터 추가

```
//1. map에 데이터 추가  
m.insert({ "aaa" , 1500 });  
m.insert({ "bbb" , 2500 });  
m.insert({ "ccc" , 600  });  
m.insert({ "ddd" , 7000 });  
m.insert({ "eee" , 1000 });
```

3) Map 데이터 전체 출력

```
//2. map 데이터 전체 출력  
cout << "1. Print Map Data\n";  
for (auto i : m)  
{  
    cout << i.first << " " << i.second << "\n";  
}
```

4) Key로 삭제

```
//3. key로 삭제
m.erase("ccc");
cout << "\n2.erase ccc\n";
for (auto i : m)
{
    cout << i.first << " " << i.second << "\n";
}
```

5) Key가 map에 존재하는지 확인하기

```
//4. Key로 찾기
cout << "\n3. find\n";
if (m.find("aaa") != m.end())
{
    cout << "find!\n";
}
else
{
    cout << "find x!\n";
}
```

6) Value로 Key 찾기

```
//5. Value로 Key 찾기
for (auto i : m)
{
    if (i.second == 7000)
    {
        cout << "\n4. value is 7000 , and what is key? , answer : " << i.first;
        cout << endl;
    }
}
```

- Value로 key를 찾는 경우는 Find의 $\log(n)$ 이점을 살리지 못하고 n 의 시간으로 찾기 때문에 좋은 방식은 아닙니다. 하지만 필요한 경우에는 사용합니다.

7) 해당 Key의 Value 변경 (map도 결국 first와 second를 사용하면 값을 변경할 수 있고 , auto를 사용하는 경우 반드시 &을 붙여야 합니다.)

```
//6. change value using key
m["bbb"] = 12000;
cout << "\n5. bbb's value changed\n";
for (auto i : m)
{
    cout << i.first << " " << i.second << "\n";
}
```

8) 해당 Value의 Key 변경 : 2단계

```
/*
7. change key
1) eliminate origin instance
2) insert new instance
*/
m.erase("bbb");
m.insert({ "BBB" , 12000 });
cout << "\n6. value : 12000's key was bbb , but now BBB\n";
for (auto i : m)
{
    cout << i.first << " " << i.second << "\n";
}
```

9) 출력결과

```
1. Print Map Data
aaa 1500
bbb 2500
ccc 600
ddd 7000
eee 1000

2.erase ccc
aaa 1500
bbb 2500
ddd 7000
eee 1000

3. find
find!

4.value is 7000 , and what is key? , answer : ddd

5. bbb's value changed
aaa 1500
bbb 12000
ddd 7000
eee 1000

6. value : 12000's key was bbb , but now BBB
BBB 12000
aaa 1500
ddd 7000
eee 1000
```

10) Find의 시간복잡도 (Set과 동일)

기본적으로 set 은 중복을 허용하지 않으며,

기본 정렬 방식은 `std::less` 이다.

set 은 red-black trees 를 기반으로 하기 때문에, 탐색, 삽입, 삭제에 대한 연산 시간복잡도가 $O(\log N)$ 이다.

2. map 안에 자료구조 넣기

1) map의 독특한 성질

하지만 set과 다르게 [] 연산자를 지원하여 insert 함수를 사용하지 않아도 쉽게 삽입을 할 수 있다.

```
map<int, int> m;  
m.insert({ 2, 1 });  
m[1] = 1;  
cout << m[1] << m[2] << "\n";
```

- 이런 방식으로 추가가 가능합니다.

2) 응용

```
map<int, vector<int>> m;  
  
m[3].push_back(1);
```

- 위의 방식을 응용하면 이렇게 가능합니다.

```
m.insert(2, 1);
```

- 이걸 안 됩니다.

3. `Unordered_map`

나. `hash_map`, `hash_set`은 표준이 아니다.

`map`과 `set`은 STL 표준 컨테이너지만 `hash_map`, `hash_set`은 표준이 아니다. 그래서 보통 STL 관련 책을 보시면 `hash_map`과 `hash_set`에 대한 설명은 없다. `hash_map`, `hash_set`을 쓰려면 라이브러리를 설치해야 할까? 그럴 필요는 없다. STL 표준은 아니지만 오래되지 않은 C++ 컴파일러에서는 대부분 지원한다. 윈도우에서는 Visual Studio.NET 이후의 모든 버전에서 지원한다.

C++11 이전 버전에서는 TR1이라는 이름으로 일부 공개했다. TR1에서는 `unordered_map`, `unordered_set`이 준비되어있다.

- C++ STL중 하나인 `unordered_map`에 대한 설명입니다.

`unordered_map`

- `map`보다 더 빠른 탐색을 하기 위한 자료구조입니다.
 - `unordered_map`은 해시테이블로 구현한 자료구조로 탐색 시간복잡도는 $O(1)$ 입니다.
 - `map`은 Binary Search Tree로 탐색 시간 복잡도는 $O(\log n)$ 입니다.
 - `unordered_map`을 사용하기 위해서는 `#include <unordered_map>` 을 선언해야 합니다.
 - `unordered_map`은 중복된 데이터를 허용하지 않고 `map`에 비해 데이터가 많을 시 월등히 좋은 성능을 보입니다.
 - 하지만, key가 유사한 데이터가 많을 시 해시 충돌로 인해 성능이 떨어질 수도 있습니다.
- Find가 $O(1)$ 이므로 Find에 매우 유리합니다.
 - Key를 기준으로 하여금 자동으로 정렬되지 않습니다.
 - Sort로 정렬이 불가능합니다.
 - 동일한 방식으로 `unordered_set`이 존재합니다.

This is impossible from both a compilation and logical standpoint. From a type standpoint, `std::sort` requires:

- RandomIt must meet the requirements of ValueSwappable and RandomAccessIterator.
- The type of dereferenced RandomIt must meet the requirements of MoveAssignable and MoveConstructible.

The iterator type on `std::unordered_map` is a ForwardIterator, not a RandomAccessIterator, so the first requirement is unsatisfied. The type of the dereferenced iterator is `pair<const Key, T>`, which is not MoveAssignable (can't assign to `const`), so the second requirement is also unsatisfied.

From a logical standpoint, sorting an **unordered** container makes no sense. It's unordered. And the complexity guarantees that `unordered_map` is able to achieve require a very specific ordering that you shouldn't be, and aren't, allowed to mess with.

If you want to "sort" your `unordered_map`, put them in a `vector`:

원소의 개수가 많을수록 해시충돌이 일어날 확률도 높아지므로

- 원소의 개수가 적고 빠른 성능을 원할 땐 `unordered_map`
- 원소의 개수가 많아 안정성을 원할 땐 `map`

- Reference : https://en.cppreference.com/w/cpp/container/unordered_map

4. Unordered_Map을 깊게 이해하기 위한 Hash Table의 개념

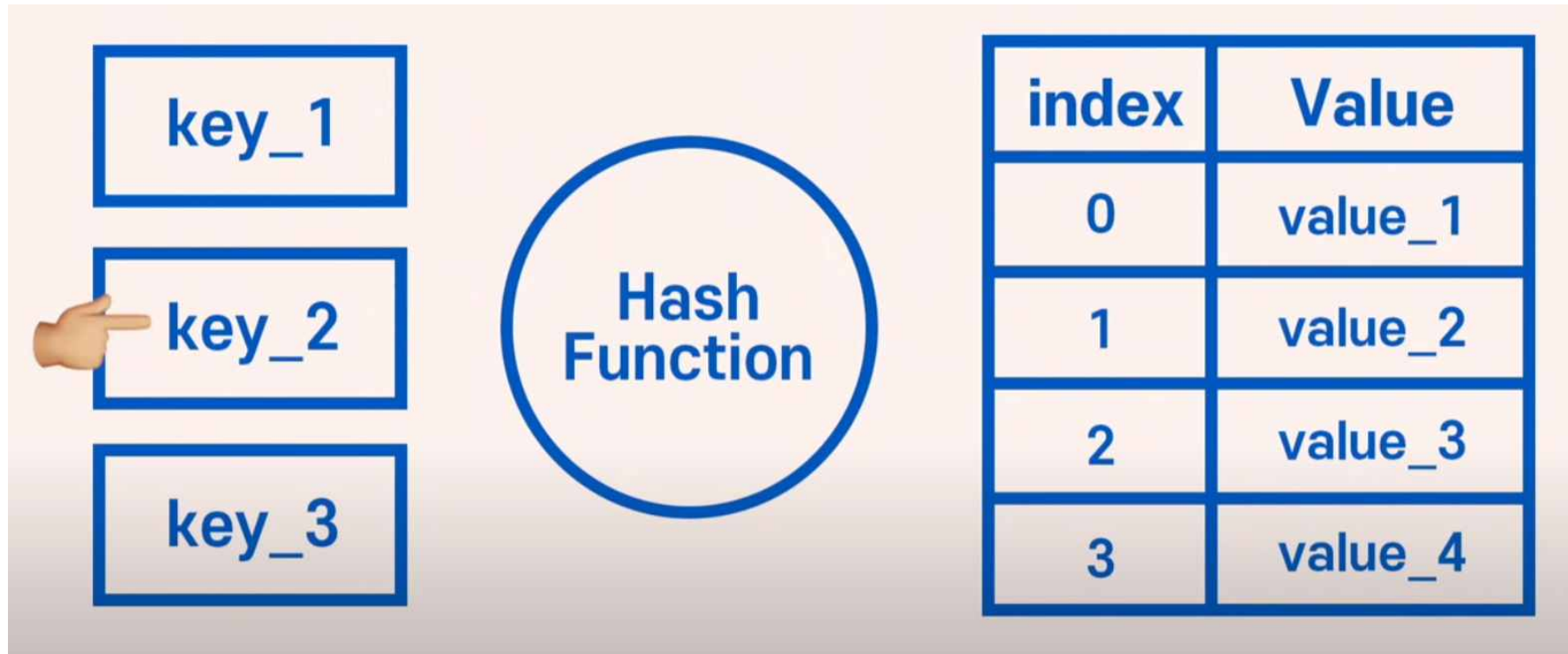
*** C++에서는 Unordered_set 과 Unordered_map으로 STL에 존재하고 있으며 , 아래의 내용은 STL의 근간인 Hash에 대한 설명입니다. ***

1) 정의

```
menu = {  
    coffee: 10,  
    burger: 15,  
    tea: 5,  
    pizza: 10,  
    juice: 5,  
};
```

- Map과 동일하게 Key 와 Value로 구성된 자료구조
 - 원하는 key의 value를 얻는데 걸리는 시간은 $O(1)$ 입니다.
 - Hash Table에 데이터를 추가하는데 걸리는 시간도 $O(1)$ 입니다.
- ex) pizza의 가격을 찾고 싶으면 key로 'pizza'를 검색하면 value로 '10'을 리턴 합니다.

2) 어떻게 이렇게 빠를까?



- Hash Table에는 Key , Value 말고도 내부적으로 Hash Function과 Index-Value 구조의 배열이 존재합니다.
- Hash Function은 (Key , Value)를 입력 받고 훌륭하게 개발된 Hash Function에 따라서 Index를 출력합니다.
- 출력으로 받은 Index를 Value와 연결시켜서 Index-Value 구조의 배열을 구현합니다.
- 그 결과 , 개발자가 Hash Table에 Key를 Input으로 던져 주면 Hash Function을 이용해서 연산을 해서 Index를 찾고 (여기서 1의 시간 복잡도를 사용) Index에 연결된 Value를 Output으로 Return 합니다.

3) Hash Table의 장단점

해시 테이블의 장점

1. 적은 리소스로 많은 데이터를 효율적으로 관리할 수 있다.
 1. 예) HDD, Cloud에 존재하는 많은 데이터들을 유한한 개의 해시(Hash)값으로 매핑하여 작은 크기의 캐쉬 메모리로 프로세스 관리가 가능하다.
2. 배열의 인덱스(index)를 사용해서 검색, 삽입, 삭제가 빠르다. (평균 시간복잡도 : $O(1)$)
 1. 인덱스를 사용해서 배열의 검색이 빠르다는 것은 당연한 소리이다. 하지만 삽입, 삭제는 왜 $O(1)$ 인가?
 2. 여기서의 인덱스는 데이터만의 고유한 위치이기 때문에 만약 삽입이나 삭제를 한다고 해도 다른 데이터로 채울 필요가 없다. 즉, 삽입이나 삭제할 때 데이터를 이동할 필요가 없기 때문이다.
3. 키(key)와 해시값(Hash)이 연관성이 없어 보안에도 많이 사용된다.
4. 데이터 캐싱(Data Caching)에 많이 사용된다.
 1. `get(key)`, `put(key)`에 캐시 로직을 추가하면 자주 hit하는 데이터를 바로 찾을 수 있다.
5. 중복을 제거하는데 유용하다.

해시 테이블의 단점

1. 충돌
2. 공간 복잡도가 커진다.
3. 순서가 있는 배열에는 어울리지 않는다.
4. 해시 함수 의존도가 높아진다.

4) Hash Table의 충돌을 해결하는 방법

- OS에서 배웠나? 어디서 배움 , 그거 보고 필기

5) 개발자가 원하는 어떤 아이템이 자료구조에 존재하는지 찾고 싶을 때

- 지금까지는 $O(\log n)$ 인 Set에서 Find를 통해 찾았습니다.
- 하지만 , Key에 아이템을 저장하고 , Value로 True를 저장하면 $O(1)$ 인 Hash Table에서 바로 찾을 수 있습니다.

```
나라들 = {  
  "🇸🇰": true,  
  "🇺🇸": true,  
  "🇸🇩": true,  
  "🇵🇰": true,  
  "🇲🇪": true,  
  "🇰🇷": true,  
  "🇨🇳": true,  
  "🇮🇹": true,  
}
```

- Reference : <https://www.youtube.com/watch?v=HraOg7W3VAM>

<6강_Tree>

1. Tree

= 가계도와 같은 **계층적인 구조를 표현**할 때 사용할 수 있는 자료구조

- 기존의 다른 자료구조들은 데이터를 저장하고 꺼내는 것에 초점을 두었다면 트리는 **데이터의 표현**에 초점을 두어야 합니다.

- 가지를 늘려가며 뻗어나가는 자료구조

- Tree의 노드의 개수가 N개 일 때 간선의 개수는 N-1개

- Tree는 Heap(자료구조)같은 방식처럼 배열로 구현할 수 있고 , pointer_1에 left 자식과 pointer_2에 right 자식을 저장하는 linked-list 방식으로 구현할 수 있습니다.

<기본 용어>

- 루트 노드(root node): 부모가 없는 노드, 트리는 하나의 루트 노드만을 가진다.
- 단말 노드(leaf node): 자식이 없는 노드, ‘말단 노드’ 또는 ‘잎 노드’라고도 부른다.
- 내부(internal) 노드: 단말 노드가 아닌 노드
- 간선(edge): 노드를 연결하는 선 (link, branch 라고도 부름)
- 형제(sibling): 같은 부모를 가지는 노드
- 노드의 크기(size): 자신을 포함한 모든 자손 노드의 개수
- 노드의 깊이(depth): 루트에서 어떤 노드에 도달하기 위해 거쳐야 하는 간선의 수
- 노드의 레벨(level): 트리의 특정 깊이를 가지는 노드의 집합
- 노드의 차수(degree): 하위 트리 개수 / 간선 수 (degree) = 각 노드가 지닌 가지의 수
- 트리의 차수(degree of tree): 트리의 최대 차수
- 트리의 높이(height): 루트 노드에서 가장 깊숙히 있는 노드의 깊이

2. Binary Search Tree (이진 탐색 트리)

1) Binary tree vs Binary Search tree

Binary 트리란?

- 자식을 최대 2개만 가진 트리

Binary Search 트리란?

- Binary Tree + BST invariant
- BST invariant : 왼쪽 subTree 는 더 작은 값을, 오른쪽 subTree은 더 큰 값을 가진다.
- 중복된 값을 가지고 있다면 나의 정의에 따라 valid를 정한다. BST Operation은 중복된 값을 허용하지만 보통의 경우 중복되지 않은 원소들을 갖는 트리에 관심이 있다.

Binary tree: Tree where each node has up to two leaves

```
  1
 / \
2   3
```

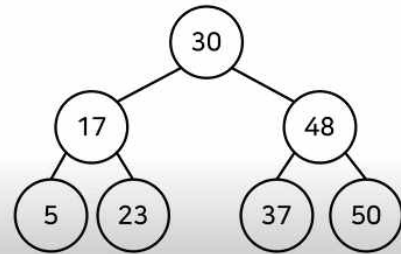
- 최대 2개이므로 자식이 0개 또는 1개여도 Binary Tree를 만족합니다.

Binary search tree: Used for **searching**. A binary tree where the left child contains *only* nodes with values less than the parent node, and where the right child *only* contains nodes with values greater than or equal to the parent.

```
  2
 / \
1   3
```

2) BST 개념

- 이진 탐색이 동작할 수 있도록 고안된 효율적인 탐색이 가능한 자료구조의 일종입니다.
- 이진 탐색 트리의 특징: 왼쪽 자식 노드 < 부모 노드 < 오른쪽 자식 노드
 - 부모 노드보다 왼쪽 자식 노드가 작습니다.
 - 부모 노드보다 오른쪽 자식 노드가 큼니다.



이제 우리는 다음을 제공하는 특정 유형의 이진 트리인 이진 검색 트리를 살펴봅니다. 특정 항목을 가능한 한 빨리 찾을 수 있도록 데이터를 저장하는 효율적인 방법입니다.

- 중복된 노드가 없어야 한다.

위 리스트는 [위키](#)에 있는 이진탐색트리에 대한 설명인데 한 가지 의문점은 왜 중복이 없어야 하는가 인데... 검색을 목적으로 하는 자료구조이기 때문에 중복이 많을 수 있는 경우 굳이 트리에 중복노드를 삽입해서 검색 속도를 느리게 할 필요가 없기 때문이다. 굳이 중복을 허용하는 이진탐색트리를 만들 필요가 있을까? 합당한 이유가 있다면 굳이 중복된 노드를 트리에 삽입하는 것 보다는 노드에 count 값을 가지게 하는 것으로 처리하는 게 효율적이라고 생각한다.

- '왼쪽 자식의 값 < 부모 노드의 값 < 오른쪽 자식 노드의 값' 성질을 이용하면 빠르게 원하는 값을 찾을 수 있습니다.
- **중복된 노드가 없어야 합니다.**

3) BST 구현

① 개념적 구현

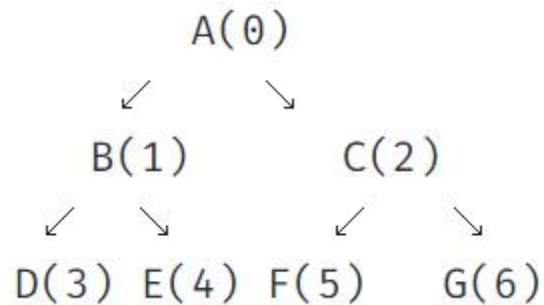
- If the given tree is empty, then simply assign the new value v to the root, and leave the left and right subtrees empty.
 - If the given tree is non-empty, then insert a node with value v as follows:
 - If v is smaller than the value of the root: insert v into the left sub-tree.
 - If v is larger than the value of the root: insert v into the right sub-tree.
 - If v is equal to the value of the root: report a violated assumption.
- 최초의 노드는 루트에 삽입합니다.
 - 그 후에 입력 값으로 들어오는 노드는 루트부터 크기를 비교하며 아래로 내려가 올바른 위치에 저장합니다.

which inserts a node with value v into an existing binary search tree `bst`. Note that the node added is always a leaf. The resulting tree is once again a binary search tree. This can be proved rigorously via an inductive argument.

- 올바르게 노드를 추가하면 무한하게 BST가 유지됩니다.

② 프로그래밍 구현 (배열)

- Vector 같은 자료구조는 STL이 존재하고 프로그래밍 구현이 간단하기에 직관적으로 이해가 갔으나 Tree는 STL도 없고, 프로그래밍 구현이 익숙하지 않았습니다.
- Tree는 무엇인가를 저장하고 꺼내야 한다는 관점으로 바라보는 자료구조가 아니라 무엇인가를 표현하는 자료구조이므로 위의 생각은 당연합니다.
- 그러나, 배열을 통해 Tree를 표현하고 구현할 수 있습니다.



루트 노드 인덱스 = 0

왼쪽 자식 노드 인덱스 = 부모 노드 인덱스 * 2 + 1

오른쪽 자식 노드 인덱스 = 부모 노드 인덱스 * 2 + 2

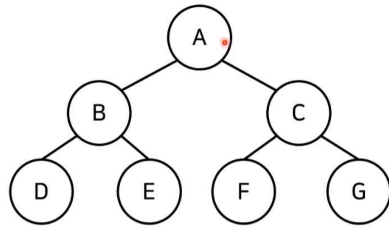
트리의 노드가 총 n개라고 하면 노드의 인덱스의 범위는 0 ~ n-1입니다.

- 다음과 같이 배열의 Index에 대한 규칙을 만들어, 해당 Index에는 어떤 노드가 들어갈지 설정하면 배열로 트리를 표현할 수 있습니다.

③ 프로그래밍 구현 (링크드 리스트)

4) BST 순회

- 전위 순회(pre-order traverse): 루트를 먼저 방문합니다.
- 중위 순회(in-order traverse): 왼쪽 자식을 방문한 뒤에 루트를 방문합니다.
- 후위 순회(post-order traverse): 오른쪽 자식을 방문한 뒤에 루트를 방문합니다.



- 전위 순회(pre-order traverse): $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$
- 중위 순회(in-order traverse): $D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G$
- 후위 순회(post-order traverse): $D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$

- 자신 방문 및 출력을 print의 P라고 합니다.
- 왼쪽 자식 방문을 Left의 L이라고 합니다.
- 오른쪽 자식 방문을 Right의 R이라고 합니다.

- 순회의 구분은 P의 위치입니다.
- 모든 순회는 L을 R 보다 먼저 하기로 약속했습니다.
- 전위 순회 = PLR
- 중위 순회 = LPR
- 후위 순회 = LRP

- 재귀적으로 순회를 천천히 생각해봅시다.

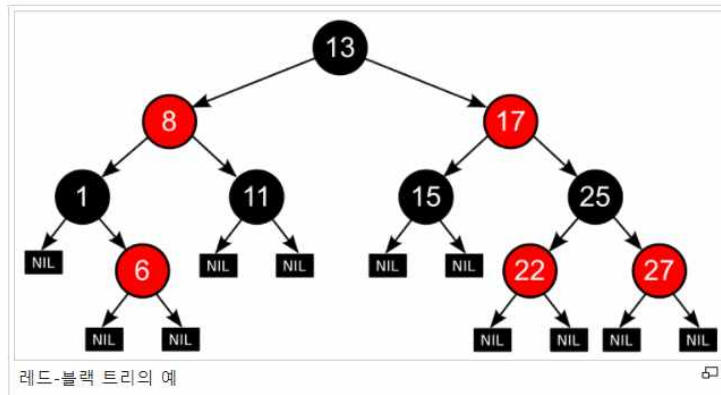
5) BST의 시간복잡도와 단점

- 평균적인 경우 $O(\log(N))$ 의 시간복잡도를 가진다.
 - 최악의 경우 $O(N)$ 의 시간복잡도를 가진다.
- 저장 순서에 따라 트리의 불균형이 발생할 수 있어 탐색의 성능에 큰 차이를 보이는 단점이 존재합니다.
 - 불균형이 심각하면 탐색의 시간복잡도가 매우 증가하여 선형탐색과 같은 $O(N)$ 의 시간 복잡도를 갖게 됩니다.

3. 자동으로 균형을 맞추는 **Balanced Binary Search Tree** (균형 이진 탐색 트리)

- BST처럼 노드를 삽입하지만 , 삽입 과정에서 자동으로 **트리의 균형을 맞추기 위해 조작을 하는 Binary Search Tree**

1) Red-Black Tree



1. 노드는 레드 혹은 블랙 중의 하나이다.
2. 루트 노드는 블랙이다.
3. 모든 리프 노드들(NIL)은 블랙이다.
4. 레드 노드의 자식노드 양쪽은 언제나 모두 블랙이다. (즉, 레드 노드는 연달아 나타날 수 없으며, 블랙 노드만이 레드 노드의 부모 노드가 될 수 있다)
5. 어떤 노드로부터 시작되어 그에 속한 하위 리프 노드에 도달하는 모든 경로에는 리프 노드를 제외하면 모두 같은 개수의 블랙 노드가 있다.

- 삽입과 삭제에 유리합니다.
- 모든 노드를 트리에 삽입할 때 루트를 제외하고는 항상 붉은 색으로 삽입합니다. 삽입할 때 마다 위의 5가지 조건 중 1개라도 만족하지 않는다면 re-coloring(노드의 색을 변경) OR re-constructing(노드의 자리 변경)을 통해 Red-Black Tree를 유지하도록 합니다.
- STL의 set과 map은 Red-Black tree로 구현되어 최악의 경우에도 $\log(n)$ 의 탐색을 보장합니다.

`std::set` and `std::map` are usually implemented as red-black trees, which are a variant of binary search trees. The specifics are implementation dependent tho.

2) AVL (Adelson-Velsky and Landis)

① 선행 개념

<Height (높이)>

- Leaf 노드의 높이는 0 이며 , Null 노드의 높이는 -1입니다.
- 부모 노드의 높이는 $\text{Max}(\text{left 자식의 높이} , \text{right 자식의 높이}) + 1$

<Balanced Factor (균형도)>

- Leaf 노드의 균형도는 항상 0입니다.
- 부모 노드의 균형도 = 좌측 자식 노드의 높이 - 우측 자식 노드의 높이
- 균형도는 부호가 존재하므로 양수라면 좌측에 쏠린 트리이고 , 음수라면 우측에 쏠린 트리가 됩니다.

② 균형도를 이용한 AVL

- 균형도를 이용하여 삽입과 삭제 시에 트리의 균형을 맞춥니다.
- 트리의 균형도가 -1 , 0 , 1 이면 균형 트리입니다
- 균형도를 유지하며 BST를 구현합니다.
- 탐색에 유리합니다.

<7. 전체적인 내용>

1. 자료구조 복사

1) 기본 문법

- 문법 : 복사를 하려는 자료구조 = 복사를 당하는 자료구조
- 일반 배열은 memcpy를 사용해서 복사합니다.

```
vector<int> v1 , v2;  
set<int> s1 , s2;  
set<int>::iterator iter;  
string str1, str2;  
for (int i = 0; i < 5; i++)  
{  
    v1.push_back(i);  
    s1.insert(i + 100);  
}  
str1 = "Study computer science";  
//복사  
v2 = v1;  
s2 = s1;  
str2 = str1;  
  
for (int i = 0; i < 5; i++)  
{  
    cout << v2[i] << " ";  
}  
cout << "\n\n";  
for (iter = s2.begin(); iter != s2.end(); iter++)  
{  
    cout << *iter << " ";  
}  
cout << "\n\n";  
cout << str2;
```

C:\ Microsoft Visual Studio 디버깅

0 1 2 3 4

100 101 102 103 104

Study computer science

2) Copy(3)

1) Question

이제 `v1`를 `v2`에 복사해야합니다. 선호 할만한 이유가 있습니까?

```
v2 = v1;
```

예

```
std::copy (v1.begin(), v1.end(), v2.begin());
```

(혹은 그 반대로도)?

2) Answer : '`v2 = v1`'만 사용합시다.

일반적으로 `v2 = v1`를 강력하게 선호합니다.

짧고 의도를 더 명확하게 만듭니다.

`std::copy`의 길이가 `v2`와 같지 않으면 `v1`가 작동하지 않습니다 (크기가 조정되지 않으므로 오래된 요소 중 일부를 유지하고 임의의 데이터를 덮어 씁니다. 프로그램의 다른 곳에서 최악의 경우에 사용

`v2.size() > v1.size()`가 곧 만료되고 C++ 11을 사용하는 경우 내용을 `v1`로 쉽게 수정할 수 있습니다.

구현자가 `move`를 사용하면 성능상의 이점이 있다면 내부적으로 `std::copy`를 사용하므로 성능 할당은 `std::copy`보다 느리지 않을 것입니다.

결론적으로 `std::copy`는 표현력이 떨어지고 잘못된 일을 할 수 있으며 더 빠르지 않습니다. 따라서 여기서 사용할 이유가 없습니다.

2. == 비교

- '==' 으로 두 자료구조가 같은지 비교할 수 있습니다.

```
queue<int> q1, q2;
if (q1 == q2)
{
    cout << "same";
}
```

3. 시간복잡도

구분	Container	시간 복잡도 (time complexity)			특징
		random access	찾기	node 삽입/삭제	
Sequence Containers	vector	$O(1)$	$O(n)$	중간 : $O(n)$ 맨 뒤 : <i>amortized</i> $O(1)$	1. 각 요소를 메모리 상 연속적 으로 저장. 2. 메모리 동적으로 할당 . 미래의 확장성을 위해 처음 생성될 때, 여유를 갖고 생성. 매번 재할당 되는 것을 방지하기 위함.
	deque	$O(1)$	$O(n)$	중간 : $O(n)$ 앞/뒤 : <i>amortized</i> $O(1)$	1. vector와 다르게 각 요소들이 메모리 상 연속적으로 저장되지 않음. 각 node는 개별적인 메모리를 할당 . 2. vector에 비해 확장하기 용이함 . (vector는 할당된 크기보다 더 커질 때, 각 요소들의 복사가 일어나는데, deque의 경우, 고정된 크기의 메모리만 할당 받고 연결시켜주기 때문.) 3. 앞/뒤 둘 다 node를 삽입할 수 있다는 장점이 있다.
	list	지원X	$O(n)$	$O(1)$	1. 구조적으로 doubly-linked list. (각 node가 개별적으로 메모리를 할당받기 때문에, 연속적인 구조에 비해 공간적으로는 비효율적일 수 있음.) 2. 특정 위치 접근은 불가능하나, 어느 위치에서든 node의 삽입/삭제가 상수 시간으로 가능함 .

Associative Containers	set	지원X	$O(\log(n))$	$O(\log(n))$	1. red-black tree 구조를 사용해서 시간복잡도를 보장함. 2. 고유한 key 값을 저장하는 컨테이너.
	map	지원X	$O(\log(n))$	$O(\log(n))$	1. 고유한 key값 - value 쌍을 이루어 저장하는 컨테이너. 2. 이것 또한 red-black tree 구조로 이루어짐.
Container Adaptors	stack			amortized $O(1)$	1. FILO(First In Last Out) 2. 시간복잡도는 해당 컨테이너에 따라감. default 컨테이너로 deque를 사용하는 경우에는 해당 컨테이너의 시간 복잡도를 따르지만, Sequence Containers 중 하나를 컨테이너로 지정할 수 있기 때문에, stack의 삽입/삭제인 push/pop은 amortized $O(1)$일 것이다.
	queue			amortized $O(1)$	1. FIFO(First In First Out) 2. stack과 마찬가지로 컨테이너에 따라 시간복잡도가 달라지지만, 기본적으로는 deque를 사용하고 있음.(앞/뒤 삽입/삭제가 많이 일어나기 때문에 vector보다는 deque나 list의 사용이 용이해 보임)
	priority queue			$O(\log(n))$	1. 가장 큰 값 or 가장 작은 값을 얻을 수 있는 컨테이너. 2. 시간 복잡도는 사용하는 container에 따라 달라질 것 같긴 하다. 3. priority queue에 대해서는 많은 의견 부탁드립니다.(default container가 vector로 되어있는지 아직 이해가 되지 않습니다.)

- 세부적인 시간 복잡도 : <https://alyssaq.github.io/stl-complexities>

4. 중첩 자료구조 (Nested)

1) 개요

```
struct str
{
    int lv;
    int cnt;
    vector<pair<string, int>> person;
};
vector<str> v;
```

```
v.push_back({ 3,5 }); //이것만 해도 v에 원소가 추가되지만 v[0]의 person 벡터는 빈 벡터 상태
//불가능 : v.push_back({3,5,{"test_1" , 11}});
v[0].person.push_back({ "test_1", 11 });
v[0].person.push_back({ "test_2" , 22 });

for (auto i : v)
{
    cout << i.lv << " " << i.cnt << " ";
    cout << "\nnested vector's instances : ";
    for (auto j : i.person)
    {
        cout << j.first << " " << j.second << " ";
    }
}
```

```
3 5
nested vector's instances : test_1 11 test_2 22
```

- 자료구조 안에 자료구조를 넣는 형식은 매우 빈번하게 사용됩니다.

- 좌측의 작성한 코드처럼 중첩된 자료구조를 구현합니다.

- 출력할 때 ranged for문과 auto 그리고 이중 for문을 사용하면 쉽게 자료구조 전체 자료를 탐색할 수 있습니다.

2) 중첩 자료구조에서 포인터로 삭제가 가능할 것 같지만 불가능합니다.

```
vector<int> arr[5];

//using pointer to clear -> X
for (int i = 0; i < 5; i++)
{
    for (int j = 0; j < 3; j++)
    {
        arr[i].push_back(i+j);
    }
}

for (int i = 0; i < 5; i++)
{
    for (auto j : arr[i])
    {
        cout << j << " ";
    }
    cout << "\n";
}

arr->clear();

for (int i = 0; i < 5; i++)
{
    for (auto j : arr[i])
    {
        cout << j << " ";
    }
    cout << "\n";
}
```

- 자료구조 Array 안에 Vector가 저장되어 있습니다.
- Array 전체를 비우기 위해 arr->clear()를 하면 될 것 같지만 불가능합니다.
- 반복문으로 array 원소의 vector를 하나씩 clear 해줘야 합니다.